PAL Manual

This manual starts with a small working model and then explains how PAL's pieces fit together. Use the API Guide for compact behavior summaries and the API Reference for signatures. A PDF copy of this Manual is also included for local/offline reading.

1. The model loop

A PAL model commonly has three parts: **Setup** creates the initial state, **Step** updates it, and **Draw** renders it. PAL does not require a model class or impose a scheduler; you choose the update order and when to render.

```
import PythonAutomataLibrary as pal

@pal.njit(cache=True)
def Step(pop: pal.PopGrid):
    # Apply a synchronous increment to every site.
    for i in range(len(pop)):
        pop.Add(1, i)
    pop.Update()

def main():
    pop = pal.NewPopGrid((40, 40))
    for _ in range(10):
        Step(pop)
    print(pop.GetPop())

if __name__ == "__main__":
    main()
```

The grid contains 1,600 sites. Each site receives one increment per timestep for 10 timesteps, so every site ends at 10 and `GetPop()` returns `16000`. `Add` queues changes; `Update` commits them together. This separation prevents an update at one site from changing the state read at another site during the same timestep.

PAL uses **safe mode** by default. Develop and validate your model in this mode. If profiling shows that validation overhead matters, call `pal.FastMode()` before creating any PAL objects to remove selected performance-sensitive checks.

2. Choose the right representation

Choose the representation that matches the state your model needs to preserve. A `Grid` stores one typed value per site; an `AgentGrid` stores identifiable agents and can hold per-agent properties; a `PopGrid` stores integer counts without individual identities; and a `PDEgrid` stores a scalar field with transport operations. A model can combine them-for example, agents can consume a `PDEgrid` field while a separate `PopGrid` tracks counts.

Use a tuple of dimensions when constructing a spatial grid. Negative dimensions mean periodic wrapping on that axis, so `(-40, 40)` wraps x but not y. `AgentGrid` also supports an empty dimension tuple for a nonspatial population.

Choosing a representation by scale

Use individual agents when identity, history, or per-agent properties affect behavior. If individuals are interchangeable within each site, a `PopGrid` can replace many agent handles with one integer count per site, often reducing update cost. Its optional `capacity` limits each site, not the total population. Use a `Grid` for nonpopulation site attributes such as terrain or a fixed mask, and a `PDEgrid` for a scalar field that changes through diffusion or advection. A hybrid model can combine representations, but must define how they exchange state.

Choose a representation according to what the model must preserve, not just the number of sites. Replacing agents with counts discards individual identity and history; replacing counts with a continuous field can discard integer and stochastic behavior. These are modeling assumptions, not merely implementation details.

3. Coordinates and neighborhoods

A grid has integer lattice sites and may also store agents at continuous positions. A **site index** `i` is an integer identifying a lattice cell; lattice coordinates such as `(x, y)` identify that cell on each axis. An agent's continuous coordinates are different: lattice site `(x, y)` spans continuous positions `[x, x+1) x [y, y+1)`, and its center is `(x+0.5, y+0.5)`. Thus an agent created or moved with `NewAgentSQ(x, y)` or `MoveSQ(..., x, y)` has integer site coordinates `XSQ=x`, `YSQ=y`, but continuous coordinates `X=x+0.5`, `Y=y+0.5`. An agent placed with `NewAgent` or `Move` instead keeps the supplied continuous coordinates while its `XSQ`/`YSQ` values identify the containing lattice site. In 2D, `ToI(x, y)` converts lattice coordinates to a linear site index, and `ItoX(i)` and `ItoY(i)` convert that index back to lattice coordinates. Do not confuse the linear site index, lattice coordinates, and continuous agent coordinates. Grid geometry is exposed as attributes such as `xDim`, `yDim`, `nDims`, and `wrapX`, not methods.

PAL provides `MooreHood`, `VonNeumannHood`, and `CircleHood`. They return relative integer offsets. `grid.Hood(hood, x, y)` maps those offsets to **lattice coordinates** (a scalar `x` in 1D, coordinate tuples in 2D/3D), wrapping periodic axes and omitting sites outside nonperiodic axes. It preserves the order and duplicates of the supplied offsets. `grid.Box(...)` instead iterates coordinates in a half-open rectangular region; in 2D it also yields `(x, y)` tuples, not linear indices. A box wider than a wrapped axis can also visit a site more than once. This distinction matters when using the results for indexing versus coordinate-based operations.

4. Individual agents

An agent is an integer handle owned by an `AgentGrid`, distinct from both its lattice site and its continuous position. Create it with `NewAgentSQ` for a lattice site or `NewAgent` for a continuous position. `NewAgentSQ` and `MoveSQ` accept either a linear site index or lattice coordinates; `NewAgent` and `Move` use continuous coordinates. An agent can move within a site without changing its lattice site. Remove it with `Dispose`.

`agents.All()` returns a snapshot of living handles, so creating or disposing agents does not change which handles are in that snapshot. On a nonstackable grid, safe mode rejects attempts to place two agents at the same site. Use `isStackable=True` only when multiple occupancy is part of the model.

Iterating nearby agents

Use `AgentsAt(x, y)` to visit the agents occupying one lattice site. For a continuous search around a point, `AgentsInRadius(rad, x, y)` yields `(agent, dx, dy, distSq)` in 2D. The displacement components account for periodic wrapping and `distSq` is squared distance. In 1D the tuple is `(agent, dx)`; in 3D it is `(agent, dx, dy, dz, distSq)`. Pass `exclude=agent` to omit an agent from its own neighborhood search.

Both iterators work in Python and inside `@pal.njit` loops. Python iteration materializes matching handles first, but the grid must not be structurally modified during that iteration. Safe mode checks for structural changes; fast mode omits this check. If the loop creates or disposes agents, iterate over a snapshot from `agents.All()` instead.

For example, this Python-side search finds the agent across a periodic `x` boundary:

```
import PythonAutomataLibrary as pal

agents = pal.NewAgentGrid((-10, 10))
agent = agents.NewAgent(9.0, 2.0)
nearby = list(agents.AgentsInRadius(1.5, 0.0, 2.0))
assert len(nearby) == 1
found, dx, dy, distSq = nearby[0]
assert found == agent
assert dx == -1.0 and dy == 0.0 and distSq == 1.0
assert list(agents.AgentsInRadius(1.5, 0.0, 2.0, exclude=agent)) == []
```

Agent birth/death timestep

`Examples/Agents/SaveLoadModel.py` is a complete agent-based model using a nonstackable 2D `AgentGrid`. Its compiled `Step` iterates `grid.All(shuffle=True)`, removes an agent on a death draw, and otherwise attempts reproduction on a second draw. To reproduce, it scans the Moore neighborhood, stores empty site indices in a reusable `IList`, and creates an agent at a randomly selected empty site.

The distinction between **snapshot membership** and **current occupancy** is important. `All()` determines which agents get a turn at the start of the step, so newborns do not act until the next step. However, each parent's neighborhood query sees the current grid; births and deaths earlier in the step can therefore change later agents'

options. Shuffling changes the update order and can change outcomes even from the same initial occupancy. This is a sequential stochastic update, unlike the queued, simultaneous `PopGrid.Add`/`Update` pattern described below.

The example's `Setup`, `Step`, and `Draw` functions are `@pal.njit(cache=True)` functions. The Python `main` loop owns the window, timestep, and checkpoints; compiled functions handle repeated model operations. Use the source example for the complete runnable program rather than duplicating its graphics and checkpoint lifecycle here.

5. Simultaneous population and field updates

`PopGrid` and `PDEgrid` distinguish current state from pending changes. Direct indexing changes the current state immediately; `Add` queues a delta, and `Update` applies pending changes. `Reset` clears both current state and pending changes.

A `PDEgrid` also provides diffusion and advection. Set the timestep and spatial spacings with `SetTimeSpaceStep(dt, dx, dy, dz)` before transport operations. Cartesian, interface-based, masked, ADI, and radial diffusion variants are available; choose the method that matches the model's geometry and numerical assumptions. Cartesian boundaries may be uniform or vary across a face; wrapped axes are periodic instead. Validate timestep stability and boundary behavior in safe mode before optimizing.

Advection versus diffusion example

`Examples/Diffusibles/ReactionDiffusion2D.py` compares two `PDEgrid` fields on the same 20 x 10 lattice. Each timestep sets a short vertical source at `x=10` in both fields. One field then calls `Advection(vx=0.01, vy=0.01, xMaxBC=0, yMaxBC=0)`; the other calls `DiffusionADI(0.01, xMaxBC=0, xMinBC=0)`. Each transport call is followed by its own `Update()`.

The example illustrates two different processes: advection transports the field with a velocity, whereas diffusion spreads it down concentration gradients. Boundary values also enter the two methods differently. Diffusion uses the boundary gradient; advection uses a boundary value only for inflow and uses the interior upwind value for outflow. The explicit boundary arguments are part of the model, not cosmetic rendering settings. `Draw` maps each field's current values through `ColorScale` into separate pixel windows. As in the agent example, `Step` and `Draw` are compiled, while `main` manages windows and iteration.

This example is qualitative, **not** a convergence or mass-conservation test. It continually resets source cells and uses different boundary conditions for the two fields. For quantitative validation, remove the source, use equivalent boundary conditions, and check mass balance and convergence across grid spacings and timesteps.

A hand-checkable field gradient

`Examples/Diffusibles/Gradient2D.py` provides a simpler numerical check than a transport simulation. It assigns `grid[x, y] = y / grid.yDim` on a 10 x 10 `PDEgrid`. The centered finite differences at `(5, 5)` are therefore

```
import PythonAutomataLibrary as pal

field = pal.NewPDEgrid((10, 10))
for x in range(10):
    for y in range(10):
        field[x, y] = y / 10

dx = (field[6, 5] - field[4, 5]) / 2
dy = (field[5, 6] - field[5, 4]) / 2
assert abs(dx) < 1e-6
assert abs(dy - 0.1) < 1e-6
```

The field is constant in x and linear in y, so its expected discrete gradients are 0 and 0.1. The assertions allow floating-point storage and arithmetic error. This verifies indexing and a finite-difference calculation without introducing boundary conditions or timestep integration. `Examples/Diffusibles/Gradient2D.py` performs the same calculation inside `@pal.njit`.

Numerical update order

Update order is part of the model. If births and deaths in a `PopGrid` must be simultaneous, queue all changes before calling `Update()`. Reads during this phase see the same starting population. Direct assignment to `pop[x, y]`, by contrast, changes what later calculations read. Mixing immediate assignments and queued changes is valid, but the order then affects the model.

For a `PDEgrid`, transport routines accumulate changes that become current at `Update()`. Check timestep stability and convergence by varying the timestep and grid spacing. Safe-mode checks do not replace numerical validation.

Consider two sites with populations 10 and 0. If a transfer moves 5 from site 0 to site 1, queue -5 and +5 before a single `Update()`; the result is (5, 5) and total population remains 10. Applying one change immediately would alter what later calculations read and can make an intended simultaneous update order-dependent.

For a hand-calculable timestep, check the required invariants: counts remain nonnegative; internal transfers conserve total population; totals change only through explicit births, deaths, or boundary flux; and a supposedly simultaneous update is independent of site visitation order.

6. Compiled model code

Decorate substantive computational functions with `@pal.njit(cache=True)` so compiled code can be reused across runs. PAL expands some model constructs before passing the function to Numba, including neighborhood iteration and source-aware safe-mode diagnostics. Annotate PAL arguments with their public types, such as `pal.AgentGrid` or `pal.PDEgrid`, for clarity and compiler support.

Keep model state in PAL objects and use `pal.Seed`, `pal.Random`, `pal.RandInt`, and `NewMultinomial` for reproducible PAL random sampling. Run a model in safe mode first. Fast mode removes selected checks; it does not make invalid indices, dead handles, or unstable numerical steps valid.

Testing a timestep

Test a model's update function on a grid small enough to inspect by hand, including an edge or wrapped boundary rather than only interior sites. Use invariants appropriate to the object: closed diffusion conserves mass within floating-point tolerance; agent moves preserve valid live handles and occupancy rules; population updates satisfy the conservation and nonnegativity checks from section 5.

When a model uses both Python orchestration and compiled `@pal.njit` updates, test both paths; passing in Python does not establish compiled behavior. Include safe and fast runs in regression testing, but use safe-mode exceptions to diagnose invalid operations rather than expecting fast mode to detect them.

Random streams and repeatability

`pal.Seed(seed)` initializes PAL's random stream. `pal.Random()` draws a uniform value and `pal.RandInt(max)` draws an integer from `[0, max)`. PAL random draws from Python and compiled PAL code consume the same stream, so **the order of calls matters**. Repeating a run requires the same seed, initial state, model parameters, and random-call order; merely reseeding halfway through a simulation does not reconstruct the prior model state.

For comparisons between two algorithms, seed each run separately and record the seed. If one implementation makes additional random draws, subsequent results can diverge even when both start from the same seed. External random libraries (such as NumPy's generators) have independent state and must be seeded or checkpointed separately.

7. Drawing and output

`StartPixWindow(xDim, yDim, scale=1, title='PAL', headless=False)` returns `(pix, window)`. Write packed RGB colors such as `0xFF0000` to `pix[x, y]`, then call `window.Update()` to publish the frame. `window.Save(path, block=True)` waits for the image to be written; use `window.Close()` to release resources. The pixel buffer also supports linear indices, slices, and writes from compiled code. For off-screen images, use `headless=True` (requires the relevant output dependencies). GIF recording uses `window.StartGif(path, delay=100)`, `window.AddGifFrame(block=False)`, and `window.StopGif()`; update the window before capturing each frame.

`StartOpenGLWindow` provides geometric 2D/3D drawing rather than a writable pixel lattice. Use its drawing object for shapes and scene controls; its window uses the same update, image/GIF output, and close lifecycle described above. `headless=True` is available for off-screen rendering but still requires a working OpenGL backend. See the API Guide or Reference for the drawing-method inventory.

Rendering and simulation are separate. A model can run without a window, draw every timestep, or draw only selected checkpoints.

8. Checkpointing and reproducibility

PAL's native-backed model objects support Python pickling, so a long-running model can save its state and resume without retaining native pointers from the original process. A checkpoint must include **all** state needed for continuation: every grid, relevant parameters, the current timestep, and any additional state managed by the model. Saving only one grid does not preserve a coupled simulation. Keep the code version and parameter configuration alongside the checkpoint so that results can be interpreted later.

A PAL object checkpoint does **not** serialize PAL's random stream. If exact stochastic continuation matters, the model must reconstruct the intended random state separately; external RNGs such as NumPy's are independent as well. Verify checkpoint behavior with a short uninterrupted run and a checkpoint-and-resume run rather than assuming object state alone reproduces the same future random draws.

A minimal checkpoint can be saved and resumed without opening a window:

```
import pickle
import PythonAutomataLibrary as pal

pop = pal.NewPopGrid((5,))
pop[1] = 10
pop.Add(7, 1)  # Pending change is part of the checkpoint.
state = {"pop": pop, "step": 3}
checkpoint = pickle.dumps(state)

restored = pickle.loads(checkpoint)
restored["pop"].Update()
assert restored["step"] == 3
assert restored["pop"][1] == 17
```

`Examples/Agents/SaveLoadModel.py` serializes a dictionary containing an `AgentGrid`, an `IList`, and the timestep, then restores all three before continuing. PAL object checkpoints do not include unrelated Python state or external RNG streams.

9. How PAL works: architecture and design decisions

This chapter explains the path from a line of model code to a state change in PAL. It is intentionally more architectural than the API chapters: the goal is to explain the responsibilities of each layer, why the boundaries exist, and what those boundaries mean when debugging, extending, or benchmarking a model.

The discussion distinguishes documented contracts from engineering rationale. The public API and tests establish what users can rely on. A rationale explains the tradeoff a design serves; it does not claim an undocumented historical motivation or that no alternative design is possible.

9.1 The design problem PAL is solving

PAL targets models that repeatedly update many small pieces of state. A timestep might inspect thousands of neighborhoods, move individual agents, update a population count at every site, solve a field equation, or combine several of these operations. Writing every inner operation as a Python call is expressive but can impose substantial interpreter overhead. Writing the whole model in C could reduce that overhead, but would make iteration and experimentation less convenient for Python users.

PAL's architecture splits the problem by frequency and responsibility. Python remains the control language for constructing the model, selecting parameters, loading inputs, managing experiments, and deciding when to draw or checkpoint. Repeated computation is expressed in functions that PAL can prepare for Numba compilation. The native core supplies low-level state and kernels behind that interface.

This split does not mean that all Python code is slow or that every model function should be compiled. It puts optimization effort where repeated work makes it worthwhile. A function called once during setup usually needs less attention than a loop executed at every site over millions of timesteps. Start with clear model code, identify the repeated work, and benchmark before moving more logic into compiled kernels.

9.2 The layers and their responsibilities

A useful mental model is to separate PAL into several cooperating layers:

1. **Public API and Protocol declarations.** These define the names, signatures, type-facing interface, and expected user-visible operations. They support readable calls, type annotations, and autocomplete. 2. **Python constructors and wrappers.** These create objects, translate public arguments into implementation calls, and expose native-backed state through a Python-friendly interface. 3. **Numba-compatible wrappers and overloads.** These let compiled model functions call PAL operations while preserving supported argument forms and return behavior. 4. **The `pal.njit` preparation step.** This transforms selected PAL-specific constructs before compilation, including neighborhood/box iteration and some source-aware diagnostics. 5. **The native C core.** This provides low-level storage and computational routines used by the Python/Numba-facing layer. 6. **Tests, examples, and generated documentation.** These act as executable and written statements of the public contract, checking that the layers agree.

These are responsibilities, not necessarily six separate packages; several may share a source file. Keeping them conceptually distinct helps localize defects. A constructor bug, a Numba typing error, a native-state problem, and a stale API declaration require different fixes.

9.3 Why the native core is behind a Python interface

The native core gives PAL a place to implement low-level operations without requiring users to write C for ordinary model logic. Native-backed objects can hold state in a representation suited to repeated updates, while the Python-facing API offers recognizable constructors, indexing, and methods.

The wrapper is important because the native implementation is not itself the public contract. A C function may accept a particular pointer or integer layout internally, but the Python API should expose a stable and understandable operation. The wrapper layer is where public arguments, supported types, safe-mode validation, and implementation details meet.

This separation creates a maintenance obligation: behavior must remain coherent across the wrapper and native layers. If the Python method accepts a keyword argument but its Numba overload silently binds that keyword differently, the layers disagree. If a native routine handles a one-dimensional coordinate differently from the safe wrapper, a mode-specific defect can result. PAL's parity tests therefore compare outcomes across paths rather than assuming that one layer automatically inherits the correctness of another.

9.4 Native-backed state and object lifetime

A PAL object is more than a Python container. Its public methods operate on state managed through PAL's native-backed implementation. The Python object is the user's handle to that state; it is not safe to assume that every internal value is an ordinary Python list or NumPy array that can be edited freely.

This matters for lifetime and mutation. Operations such as creating, moving, or disposing an agent must update the grid's internal structures consistently. Writing directly into an internal count or pointer-like representation could bypass the rules that keep occupancy and agent handles aligned. PAL exposes explicit methods for state changes so the implementation can preserve those invariants.

The same distinction explains why a returned array or slice must be interpreted according to its documented contract. A detached copy is a snapshot, not a live view into native memory. Editing it changes the copy until the result is explicitly assigned back. This is less magical than pretending every slice is a mutable view and makes ownership clearer at the Python boundary.

Native-backed objects can support pickling, but pickling an object is not equivalent to serializing the entire simulation process. A model's complete checkpoint may also need parameters, timestep, other coupled objects, external state, and random-number state. See section 8 for the checkpoint contract and its limitations.

9.5 The public API is a contract, not a mirror of the C implementation

PAL's public names and signatures are designed around modeler-facing concepts: a grid site, an agent handle, a population count, a neighborhood, a field, or a random draw. The native core may use lower-level representations that do not resemble those concepts directly. That is acceptable as long as the public behavior remains consistent.

Protocol declarations are part of this contract because users and tools read them for annotations and autocomplete. But a declaration is not executable proof. Likewise, a docstring or manual paragraph cannot make unsupported runtime behavior valid. The actual implementation, declarations, tests, and generated references need to describe the same public surface.

This is why API changes should be made as a coordinated change. When altering an argument name, return type, indexing rule, or safe-mode behavior, check the Python method, the Numba overload, relevant Protocol declaration, documentation generator, examples, and regression tests. A change that works in one path but breaks another is not complete merely because the primary example still runs.

9.6 Why PAL has both Python and Numba execution paths

Python execution is useful for interactive exploration, setup code, inspection, and small operations. Numba-compiled execution is useful for repeated loops where Python dispatch would be costly. PAL supports both because a scientific workflow naturally mixes these kinds of work.

Supporting both is not free. Numba imposes typing and compilation constraints that ordinary Python does not. Native-backed objects need compatible wrappers or overloads; argument types and names must be represented in forms Numba understands; and code transformations must produce valid compilable functions. An operation can therefore work in Python and fail at compile time, or compile successfully but behave differently in an edge case.

PAL treats parity as a correctness requirement for supported usage, not as a promise that Python and compiled code have identical internals or performance. Compare observable behavior: return values, mutations, errors where they are part of the contract, boundary handling, and state after the operation. If a difference appears, reduce it to a minimal reproducer and test both paths.

9.7 Why `pal.njit` transforms model code

PAL's `njit` interface is not simply a spelling alias for Numba's decorator. Before compilation, it can prepare functions that use PAL-specific syntax and diagnostics. In particular, neighborhood and Box iteration are model-friendly abstractions that may need to be expanded into forms that the compiler can type and execute efficiently.

The transformation lets users express spatial operations directly instead of writing coordinate conversions in every loop. It also gives PAL a place to add source-aware safe-mode diagnostics when argument annotations provide enough type information.

Transformations add a layer that users should understand when debugging. The Python source a modeler reads is not necessarily the exact function body Numba receives. A traceback or typing error may refer to transformed code or a synthetic compilation context. Annotations improve the information available to the transformer, but they do not guarantee that every error can be attributed to a perfect source location.

When a failure occurs only under `@pal.njit`, do not immediately rewrite the scientific algorithm. First isolate the smallest function that reproduces it, confirm argument annotations and supported API usage, and compare it with a Python version. Then determine whether the problem lies in the transformation, Numba typing, a wrapper, or the underlying operation.

9.8 Why neighborhoods are represented as relative offsets

A neighborhood such as Moore or Von Neumann is naturally a pattern around a location. PAL represents it as relative integer offsets, then applies it to a specific grid position with `grid.Hood(...)`. This separates the reusable pattern from the grid location where it is used.

The distinction also makes boundary behavior explicit. Applying a hood must account for grid dimensions and wrapping flags; the relative pattern alone does not know whether a particular neighbor falls outside a nonperiodic domain. PAL's application step can translate offsets into coordinates, wrap periodic axes, and omit out-of-domain sites according to the public contract.

This avoids storing a separate absolute-neighbor list for every cell and lets the compiled path transform high-level neighborhood loops. It does not guarantee unique destinations: small or wrapped domains can map multiple offsets to the same site. Whether to count those repeated contributions or deduplicate them depends on the model, so choose and test the intended behavior.

9.9 Why update and commit are separate operations

PAL makes a distinction between changes that are being accumulated and changes that have become the current state. For `PopGrid` and `PDEgrid`, `Add` queues a delta and `Update` commits pending changes. This makes the timestep boundary visible in code.

This distinction matters mathematically. If every new site value must be computed from the same old field, immediate mutation can make results depend on iteration order. Queued deltas let the model calculate changes before committing them. Direct indexing remains available when immediate assignment is intended; PAL permits order-dependent algorithms but makes the choice explicit.

Agent updates have different semantics. `All()` snapshots which handles will be visited, but it does not snapshot the world. If one agent reproduces, later agents may see the newborn's occupancy even though the newborn is not part of the original handle snapshot. This is a deliberate and useful pattern for sequential agent models, but it is not equivalent to a synchronous cellular update.

PAL does not impose one scheduler because these update schemes represent different models. Choose the intended timing and test it with a small case whose result can be calculated by hand.

9.10 Why safe mode and fast mode are process-wide

Safe mode performs selected validation and provides diagnostics that are useful while developing a model. Fast mode removes selected checks from performance-sensitive paths. Keeping one mode active for a process makes the execution contract predictable for objects created within that process; PAL locks the choice at first construction.

A process-wide mode also avoids pretending that an individual object can be switched independently when calls across objects may share compiled functions or native machinery. The consequence is operational: call `pal.FastMode()` before creating any PAL object or window, and run safe and fast test suites in separate processes.

Fast mode should be treated as a measured optimization, not a correctness mode. It can reduce validation overhead; it cannot repair invalid model assumptions, unsafe numerical parameters, or a wrong update order. Safe mode can catch selected contract violations, but it cannot establish that the biological or physical equations are appropriate.

9.11 Why visualization is not part of the simulation kernel

PAL separates simulation state from rendering. The model changes its grids and agents; the drawing layer turns the current state into pixels or geometric primitives. This permits the same model to run headlessly, draw every timestep, draw intermittently, or save output without changing the scientific update rules.

The split is important for performance. Rendering can be expensive and can obscure the cost of the simulation itself. When benchmarking, measure the update kernel separately from drawing, then measure end-to-end runtime if that is what users experience. When validating a model, avoid treating a plausible-looking image as proof of correct state transitions.

The two main window abstractions serve different needs. A pixel window is useful when state maps directly to a pixel lattice; an OpenGL window supports geometric 2D/3D drawing. Headless modes enable automated image/output workflows where supported, but they do not test every interactive display behavior or guarantee that every graphics backend is available on every machine.

9.12 Why random-number state is a separate concern

PAL's random functions expose an explicit stream that model code can seed and consume from Python or compiled functions. This supports repeatable experiments when the initial state, parameters, seed, and order of random calls are held fixed.

The random-call order is part of reproducibility. Adding a draw to one branch changes the state of the stream and may change all subsequent random outcomes. Consequently, two versions of an algorithm can use the same seed but produce different trajectories because they consume random values differently. This is expected, not evidence that seeding failed.

Randomness is also separate from model-object state. Saving a grid or agent structure does not automatically preserve every random generator used by the process. Exact stochastic continuation requires explicit treatment of PAL's stream and any external RNG state. Record seeds and configurations for repeatable experiments, and do not promise bit-for-bit continuation from an object checkpoint alone.

9.13 Why testing is layered

No single test style can establish every kind of correctness. PAL uses several complementary forms of evidence:

- **Focused API tests** check individual public operations, argument forms, boundary cases, and error behavior. - **Python/Numba parity tests** check that supported calls agree across execution paths. - **Safe/fast tests** check behavior under both validation settings, which must be initialized in separate processes. - **Mathematical invariants** check properties such as conservation, coordinate identities, nonnegativity, or known gradients. - **Integrated examples** exercise multiple components together under realistic model control flow. - **Build and CI checks** rebuild the native core and run the suite against the repository source, reducing the risk of accidentally testing a stale binary or incomplete local setup.

Each layer catches a different class of defect. A focused test localizes a regression; a parity test catches path divergence; an invariant can catch an error shared by multiple implementations; and an integrated model can expose interaction bugs that isolated unit tests miss. Passing these tests still does not prove that a scientific hypothesis is correct, so model validation remains a separate responsibility (section 14).

9.14 Why performance claims require more than one timing

PAL includes compilation, native calls, validation, model logic, and sometimes rendering in one user workflow. A timing that mixes these costs can be misleading. The first call to a compiled function may include compilation and specialization; later calls measure steady-state execution. A benchmark that measures only the first call may primarily be measuring compilation.

For useful comparisons, separate cold compilation from hot runtime, use the same workload and state, repeat measurements, and check outputs for equivalence. Test the safe and fast modes in separate processes. Include the dimensions, population size, neighborhood size, number of timesteps, and whether rendering is enabled. A microbenchmark can reveal a local cost, but the actual model workload decides whether that cost matters.

The design goal is not to maximize every isolated benchmark at the expense of a comprehensible API. A performance change should be accepted when it is reproducible, preserves behavior, and improves a representative workload. Prefer algorithmic reductions in unnecessary work before complicating code for a tiny or noisy gain.

9.15 How to trace a PAL operation when debugging

When an operation behaves unexpectedly, follow the layers in order:

1. **Check the public contract.** Confirm the method, argument forms, coordinate convention, and return type in the API Guide and Reference. 2. **Reduce the model.** Build the smallest example with a tiny grid and a known expected result. 3. **Compare Python and compiled paths.** If the error appears only under `@pal.njit`, inspect annotations and transformation-related constraints. 4. **Compare safe and fast modes in separate processes.** A safe-mode exception may reveal a contract violation that fast mode does not check. 5. **Inspect the state transition.** Verify whether a method mutates immediately or queues a change for `Update()`; verify whether iteration uses a snapshot or current occupancy. 6. **Check geometry and boundaries.** Confirm dimensions, wrapping, index conversions, and neighborhood duplicates. 7. **Check independent invariants.** Test a conservation law, exact coordinate identity, or hand-computed update. 8. **Only then localize the implementation layer.** Determine whether the defect belongs to the public wrapper, Numba overload/transformer, native core, or visualization layer.

This sequence moves from observable contract to minimal evidence before diving into implementation details. It reduces the risk of "fixing" a wrapper symptom by changing the model's scientific behavior.

10. Testing PAL

PAL's `Testing/` suite exercises public behavior across geometry and wrapping, Grid/IList/Multinomial/RNG operations, AgentGrid and PopGrid state changes, PDE transport and boundaries, compiled Python/Numba parity, visualization, integrated models, and historical regressions. Many tests use independent invariants or small reference calculations rather than reproducing PAL's implementation.

Run the full suite from the repository root in separate safe and fast processes:

```
PAL_TEST_MODE=safe python -m pytest -q Testing
PAL_TEST_MODE=fast python -m pytest -q Testing
```

PAL locks its process-wide mode at first construction, so safe and fast should not be combined in one Python process. Fast mode intentionally skips some validation tests. The CI workflow runs both modes on Linux, verifies all generated documentation artifacts are current, runs the API benchmark after successful tests, and separately runs representative examples in safe and fast modes on Windows. Headless OpenGL tests require the relevant graphics

backend.

When adding or changing model code, start with the smallest test that expresses the intended public behavior, then run the affected test file before the full suite. Prefer conservation laws, coordinate identities, exact small examples, and Python/compiled agreement over tests that duplicate PAL's internal formulas.

11. Design principles: why PAL is shaped this way

This chapter makes explicit the tradeoffs that are easy to miss when reading the API as a list of methods. Some choices are direct consequences of the implementation; others are engineering priorities reflected in the public contract. They should not be mistaken for claims that every alternative was exhaustively benchmarked.

11.1 Keep model logic in Python, put repeated work in compiled kernels

PAL is designed for models whose timestep performs many small operations: inspect a neighborhood, update state, move agents, accumulate a field, or sample a random event. Python is productive for configuring experiments, loading data, controlling the simulation loop, and drawing. But calling Python for every cell or agent can dominate the cost of a large model.

PAL therefore separates orchestration from the inner loop. A normal Python function can create objects, open a window, choose parameters, and decide how many timesteps to run. Functions decorated with `@pal.njit(cache=True)` express the repeated computation. The cache can avoid recompiling the same specialization on later runs, while the first call may still pay a compilation cost.

This division is a recommendation, not a requirement that every function be compiled. Start with clear Python code, identify the repeated work, and move the computational kernel into compiled code when that is useful. Keep file I/O, interactive controls, and other orchestration outside compiled functions unless the public API explicitly supports them.

11.2 Use a small set of representations with distinct meanings

PAL deliberately distinguishes a typed `Grid`, an `AgentGrid`, a `PopGrid`, and a `PDEgrid`. These are not interchangeable containers with different names. Each representation encodes assumptions about what the model needs to remember:

- A `Grid` stores a value at each lattice site. Use it for attributes such as terrain, occupancy masks, or local state that is not itself a collection of individuals. - An `AgentGrid` preserves individual identity. Agents can have different positions, histories, properties, and fates. - A `PopGrid` stores integer counts. It is suited to interchangeable individuals when the model needs the number at each site, but not a handle for each one. - A `PDEgrid` stores a scalar field and supplies numerical transport operations such as diffusion and advection.

The point is not to force every model into one abstraction. It is to make the meaning of state explicit. A count grid avoids per-individual bookkeeping, but cannot answer which individual moved or what its personal history was. A continuous field can represent concentration smoothly, but it does not preserve integer identity or discrete events. A hybrid model is often appropriate, provided the conversions between representations are part of the scientific model rather than accidental implementation details.

11.3 Make update timing visible

A simulation's update order is part of its mathematics. PAL does not hide every update behind an automatic scheduler because different models need different timing semantics. A synchronous cellular update, a sequential agent sweep, and a numerical field solve are not the same operation.

For `PopGrid` and `PDEgrid`, queued changes and `Update()` provide a transaction-like boundary: calls that accumulate deltas do not immediately become the current state. This lets a model calculate changes from one shared starting state and then commit them together. Direct indexing, by contrast, changes current state immediately. Mixing the two is permitted, but the order then matters and must be deliberate.

An `AgentGrid` has a different contract. `All()` returns a snapshot of the living handles at the time it is called, so adding or removing agents does not change which handles are in that snapshot. It does not freeze the entire world: occupancy queries performed later still see the current grid, including births and deaths that have already happened. This is useful for a sequential agent update, but it is not a synchronous update. If all agents must act from the same old state, store the necessary old values separately and commit changes in a second phase.

11.4 Make dimensionality explicit but keep the API unified

Most model concepts-grids, neighborhoods, movement, and fields-exist in one, two, or three spatial dimensions. Requiring a separate class and method family for every dimension would multiply the API surface and make examples harder to transfer between models. PAL instead uses dimension-aware objects and coordinate tuples, with shared geometry metadata such as `nDims`, `xDim`, `yDim`, `zDim`, and wrapping flags.

The coordinate shape still matters. In one dimension a site can be represented by one integer; in two and three dimensions, coordinate tuples identify the axes. PAL retains explicit site-index operations where they are useful, but APIs that mean continuous position remain distinct from APIs that mean lattice position. This prevents a linear index, a lattice coordinate, and a continuous location from being silently treated as the same thing.

A dimensions tuple also communicates the shape of a lattice. A negative extent requests wrapping on that axis; for example, `(-40, 40)` wraps x but not y. This is compact, but it means that the sign of a dimension is configuration, not a request for a negative-sized array. When reading a model, always check both the absolute extents and the wrap flags implied by the signs.

11.5 Distinguish lattice coordinates from continuous positions

A lattice site is discrete; an off-lattice agent position is continuous. PAL keeps both concepts because many models need both: an agent may move continuously while still interacting with fields stored at lattice sites.

For a lattice site `(x, y)`, the associated unit cell occupies `[x, x+1) x [y, y+1)`. Its center is `(x+0.5, y+0.5)`. Creating an agent with `NewAgentSQ(x, y)` places it at the site center; creating it with `NewAgent(x, y)` uses the continuous coordinates supplied. The resulting `XSQ`/`YSQ` values identify the containing site, while `X`/`Y` retain continuous position.

This distinction affects neighborhood searches, visualization, coupling to fields, and boundary behavior. Use site-based movement when the model is lattice-based. Use continuous movement when sub-site displacement matters. Do not round continuous positions manually unless that rounding is part of the model; use the documented conversion or site properties.

11.6 Separate relative neighborhoods from their application

`MooreHood`, `VonNeumannHood`, and `CircleHood` describe relative integer offsets. A hood is reusable geometry: it is not itself a list of absolute sites around a particular location. Applying `grid.Hood(hood, x, y)` translates those offsets to lattice coordinates, applies wrapping, and omits coordinates outside nonperiodic boundaries.

This separation makes neighborhood definitions independent of the grid location and allows PAL to expand neighborhood loops in compiled functions. It also exposes an important detail: wrapping may map multiple offsets to the same site, and the API preserves offset order and duplicates. If duplicate visits would double-count a contribution, choose a neighborhood and domain size that avoid them or explicitly deduplicate according to the model's intended semantics.

`Box` has a different purpose: it describes a half-open rectangular region. The upper bound is excluded, so `Box(0, 4)` visits 0, 1, 2, and 3 along that axis. Neighborhood offsets are relative; Box bounds describe a region. Confusing the two produces off-by-one errors and boundary mistakes.

11.7 Make safety a process-level choice

PAL's safe mode is the default. It adds validation and diagnostics around selected operations, helping catch invalid indices, illegal occupancy changes, and other contract violations close to their source. This is especially valuable while developing a new model, when an error may otherwise appear later as a corrupted result.

Fast mode removes selected checks for performance-sensitive paths. It does not turn invalid operations into valid ones, and it does not make a numerically unstable scheme stable. PAL locks the process-wide mode when the first PAL model or window is constructed; call `pal.FastMode()` before any such construction. Because the choice is process-wide, compare safe and fast behavior in separate processes rather than trying to switch modes midway through a run.

This is an explicit tradeoff: predictable diagnostics during development, with a deliberate route to reduce selected overhead after profiling. Safe mode is not a proof of scientific correctness. A program can use every API correctly and still encode the wrong equations or assumptions.

11.8 Use a native core behind a Python-facing API

PAL's public Python API is designed to remain readable and usable from Numba-compiled model code. The native C core provides storage and kernels; Python and Numba-facing wrappers expose them to both execution paths. The wrappers connect the API to the implementation rather than maintaining a separate copy of model state.

This separation lets PAL combine Python usability with native-backed data structures and compiled loops. It also creates obligations: wrapper signatures, argument names, indexing behavior, safe-mode checks, and Python/compiled behavior must agree. A mismatch between a Python path and a compiled path is a correctness defect even if each path appears plausible in isolation.

The public Protocol declarations provide the annotation and autocomplete surface. They should describe the supported public contract, but a declaration alone is not proof of runtime behavior. The implementation and executable tests must agree with the published contract. When changing an API, update its behavior, type-facing declarations, reference material, examples, and regression tests together.

11.9 Prefer explicit state changes and inspectable contracts

PAL exposes explicit state-changing operations such as `Add`, `Update`, `Reset`, `Move`, and `Dispose`. This makes important transitions visible in the model code: a queued delta is not yet part of the committed population, and moving an agent is different from disposing its handle.

This explicitness requires the model author to understand lifecycle and update order, but it makes the simulation easier to reason about and test. For stateful models, clarity about when state changes is more valuable than hiding those changes behind a convenient-looking abstraction.

11.10 Why safe/fast and Python/compiled parity are tested separately

A model can fail in more than one way: an operation can violate the API contract; the Python and compiled paths can disagree; or both can behave consistently while the scientific model is wrong. PAL's tests try to separate these failure classes.

Safe and fast modes are tested in separate processes because mode selection is locked at construction. Python and compiled paths are compared because the public API is intended to work in both settings. Small exact examples and independent invariants-such as conservation, coordinate identities, and known gradients-are preferred where possible because they can reveal a shared implementation mistake that a test duplicating the implementation's formula might miss.

Integrated examples then exercise combinations of components under realistic control flow. No single test layer is sufficient: focused tests localize defects, parity tests catch execution-path differences, invariants check mathematical properties, and full examples reveal integration problems.

12. Subsystem design choices

The previous chapter explains cross-cutting principles. This chapter follows those principles into individual subsystems and describes the practical consequences of their contracts.

12.1 Grid: typed site state and explicit indexing

A `Grid` represents one typed value per lattice site. The element type is chosen at construction, which makes the intended state representation explicit and allows the implementation to use compact native storage rather than a Python object at every site. Use a type that matches the model's meaning: integer labels for categories, Boolean values for masks, and floating-point values for continuous attributes.

Indexing supports both linear site indices and coordinate forms, but these are different views of the same lattice. Use coordinate indexing when the equation is naturally spatial; use a linear index when iterating sites or passing a site identifier between APIs. In more than one dimension, coordinate tuples are often easier to review because the axes remain visible. The conversions `ToI` and `ItoX`/`ItoY`/`ItoZ` are explicit so the mapping can be tested rather than guessed.

Grid slices are detached NumPy copies, not live native-backed views. This avoids pretending that a slice is a mutable view into storage when it is actually materialized for Python. Read a slice as a snapshot, then assign the result back if changes should affect the Grid. In safe mode, assignment validates relevant type, range, and nonfinite conditions; do

not rely on fast mode to diagnose invalid values.

12.2 AgentGrid: handles, occupancy, and mutation

An agent handle is distinct from its position. The handle identifies the living agent; position determines where it is located. This separation lets a model retain identity while an agent moves, and lets queries report handles without copying whole Python objects. The model remains responsible for the meaning and lifecycle of any additional per-agent state.

Stackability is an explicit constructor choice because allowing multiple agents at a site changes the model's state space and affects occupancy logic. A nonstackable grid rejects a second occupant in safe mode. Do not enable stackability merely to suppress an error: decide whether multiple occupancy is biologically or physically meaningful.

The distinction between `All()` and a live iteration matters for mutation. `All()` returns a snapshot of handles and is therefore the appropriate pattern when the loop can create or dispose agents. A snapshot freezes membership, not agent state or grid occupancy. Each later query observes current occupancy. If newborns must not act until the next timestep, build the snapshot before processing the timestep; if all agents must compute from the same old world, snapshot the required state as well.

The `counts` property is a read-only occupancy query view, not a writable alternate path to agent state. Changing a count directly would bypass agent handles, placement rules, and lifecycle bookkeeping. Query occupancy through this view; create, move, and dispose agents through the AgentGrid API.

12.3 PopGrid: counts instead of individual objects

A `PopGrid` stores integer population counts per site. It is intended for populations whose members are interchangeable for the questions being asked. That can greatly reduce storage and update overhead compared with representing every individual as an agent, but it removes individual-level history and identity.

Its `Add` operation queues a delta; `Update` applies pending changes; `Reset` clears current state and pending deltas. The explicit delta buffer makes it possible to compute a set of changes before committing them. For a movement event from site A to site B, queue a negative delta at A and a positive delta at B, then update once. If the model reads current counts during this phase, it sees the pre-commit counts, which helps make the update independent of visitation order.

The `capacity` option is a per-site cap. It is not the total population limit. `All()` returns the indices of nonzero sites, not the counts themselves; read the count at each returned index when the value is needed. Keeping these semantics distinct avoids confusing the set of occupied sites with the population data stored there.

12.4 PDEgrid: numerical operations with explicit assumptions

A `PDEgrid` stores a scalar field on a lattice and supplies numerical operations for changing it. The library provides families of diffusion and advection routines because there is no single transport method that is best for every geometry and numerical regime. Cartesian, interface-based, masked, ADI, and radial methods encode different assumptions about dimensionality, geometry, boundaries, and update strategy. Choose a method because it matches the problem, not because its name sounds more sophisticated.

Call `SetTimeSpaceStep` to specify the time and spatial scales used by transport methods. These values are part of the discretized equation. A timestep that is too large can produce unstable or inaccurate results even if every API call is valid. Boundary conditions likewise belong to the mathematical problem: fixed-value, face-dependent, and periodic boundaries can produce different solutions.

PDE updates use pending changes and `Update()` to separate the calculation from committing the result. This supports a consistent old-state/new-state distinction, but does not automatically prove stability, positivity, or conservation for every scheme and parameter choice. Test the method on a known field, verify the boundary behavior, and check convergence as the timestep and grid spacing change.

12.5 IList: reusable collections for compiled workflows

`IList` is a PAL-managed integer list for workflows that need to retain, shuffle, or compose collections of indices or handles, including inside compiled code. It can be useful when building a list of empty neighboring sites before choosing one for reproduction, or when a query result needs to survive beyond a single iteration.

`All()` returns a detached copy; `Iter()` avoids creating that copy when the caller only needs to traverse the current list. That is a deliberate distinction between a convenient independent result and a lower-overhead traversal. Do not assume a returned copy is a live view. Clear and reuse a list when repeated allocation would otherwise become part of the timestep cost, while ensuring its contents are reset at the right point in the model.

12.6 Randomness: one explicit PAL stream

PAL exposes `Seed`, `Random`, and `RandInt` and provides a multinomial sampler so model code can use a consistent PAL random stream in Python and compiled code. The stream is call-order dependent: each draw advances the shared stream, so adding a draw in one branch can change all later random outcomes. Reproducibility therefore requires more than choosing a seed; it requires matching initial state, parameters, and random-call order.

Use PAL's RNG functions when a model should share one reproducible stream across PAL calls. External RNGs have independent state and need their own seed or checkpoint handling. A PAL object checkpoint does not by itself serialize the random stream, so exact stochastic continuation requires separately preserving or reconstructing random state. Do not claim that a seed alone guarantees identical outcomes after changing the algorithm.

12.7 Multinomial sampling: configuration versus sample state

`NewMultinomial` constructs a sampler for repeated multinomial or binomial sampling. Constructing one from another sampler reuses solver configuration, not the other sampler's current sample state. This separation allows a model to share sampling setup without implying that two sampler objects are the same evolving object. The safety mode must match, because PAL's mode is process-wide and sampler behavior belongs to that contract.

12.8 Visualization: simulation state versus presentation

PAL separates model state from rendering. A `PixWindow` gives the model a writable pixel buffer; it is appropriate when the simulation naturally maps one site or quantity to a pixel. An `OpenGLWindow` provides geometric drawing for 2D/3D scenes. These are different presentation abstractions, not two names for the same renderer.

For pixel rendering, write colors into the buffer and call `Update()` to publish the frame. For OpenGL, use the drawing interface to collect primitives and then update the window. In both cases, drawing frequency is a model-control decision: rendering every timestep is convenient for inspection but can dominate runtime, so long runs may draw only selected steps. Keep drawing outside the computational kernel unless a specific compiled drawing API is intended.

Headless rendering exists so output can be generated and tested without a visible interactive window, but it still depends on the relevant image or graphics backend. A headless test validates the supported off-screen path; it cannot fully validate window placement, user input, or a display server's lifecycle.

12.9 Source-aware diagnostics and AST transformation

PAL's `njit` interface transforms selected PAL constructs before Numba compiles the function. For example, neighborhood and Box iteration can be expanded in compiled code, and annotated arguments can receive source-aware diagnostics in safe mode. This helps preserve a modeler's high-level expression of a neighborhood loop while making the repeated operations suitable for compilation.

The transformation layer is also a reason to test examples and edge cases rather than assuming that valid Python syntax guarantees identical compiled semantics. The AST transformation, wrapper signatures, native core, and Numba typing rules all participate in the final behavior. When an error appears only in compiled mode, reduce it to a minimal function and compare the public behavior in Python and compiled execution.

Source-line diagnostics depend on sufficient type information, including annotations for relevant PAL arguments. Unannotated arguments do not promise the same diagnostics. Treat annotations as part of writing clear compiled model code, not as a guarantee that every error can be attributed to an exact source expression.

12.10 Why the public API and implementation are tested together

PAL's public API is intended to feel coherent across Python and `@pal.njit`, but the two paths have different execution machinery. The Python wrapper may validate an argument before calling native code, while the compiled wrapper may need a Numba-compatible overload and may use different diagnostics. Both must still agree on the public result and supported inputs.

This is why the project uses API coverage inventories, Python/compiled parity tests, integrated examples, and safe/fast test runs. A type declaration or docstring is not sufficient evidence if runtime behavior disagrees.

Conversely, a one-off implementation detail should not be elevated to a public promise unless the tests and documentation support it. When changing a contract, update the implementation, declarations, generated reference, manual, examples, and tests as one coherent change.

13. Performance: measure the right thing

13.1 Cold compilation is not steady-state runtime

The first call to a compiled function may include Numba compilation and specialization. Later calls can be much faster, particularly when caching is enabled. Timing one first call and reporting it as the simulation's runtime mixes two different costs.

For a useful benchmark, measure cold compilation in a fresh process and steady-state runtime over repeated timesteps separately. Use the same model state, workload, seed, and operation count for each comparison. Report medians over repeated runs rather than relying on one timing. If comparing safe and fast modes, use separate processes because the mode is fixed at construction.

13.2 Optimize the actual workload

A faster microbenchmark does not guarantee a faster full model. An optimization may help a large agent sweep but hurt a small grid, or reduce steady-state time while increasing compilation cost. Before changing implementation, define the workload that matters: number of sites or agents, number of timesteps, neighborhood size, mutation rate, transport method, and mode.

Use a before/after benchmark with fresh-process cold compilation and repeated hot runtime. Keep workload sizes identical, check outputs for equivalence, and investigate any reproducible regression. Treat tiny differences as noise until repeated measurements show otherwise. Prefer simpler correct code when a performance gain is not robust.

13.3 Know what fast mode cannot fix

Fast mode can reduce selected validation overhead; it cannot eliminate the fundamental cost of updating millions of state elements, choosing an inefficient representation, doing unnecessary neighborhood work, or solving an unstable numerical scheme. Before optimizing low-level code, ask whether the model's algorithm is doing unnecessary work. Replacing individually represented interchangeable agents with counts, reusing a neighborhood list, reducing repeated conversions, or drawing less often can have a larger impact than a local kernel tweak.

Do not optimize by removing a check merely because it appears in a profile. First establish that the check is responsible for a meaningful fraction of runtime, that the faster behavior is safe for the intended workload, and that tests cover the affected contract.

14. Validation: prove the model, not just the program

14.1 Start with a hand-calculable case

Before running a large simulation, reduce the domain and timestep count until the expected answer can be computed manually. A two-site transfer can establish whether queued population changes conserve the total. A linear field can establish whether indexing and finite differences produce the expected gradient. A tiny periodic grid can show exactly which neighbor lies across a boundary.

Small examples are powerful because they turn vague visual plausibility into an assertion with a known answer. Keep such examples as regression tests after the model grows.

14.2 Test invariants and edge cases

Choose invariants from the scientific model. For a closed diffusion problem, total mass should be conserved within a stated numerical tolerance. For an internal population transfer, the total should remain constant and counts should stay nonnegative. For an agent model, every live handle should refer to a living agent and occupancy should obey the stacking rule. For geometry, conversions should round-trip and wrapping should return the expected coordinates.

Test the cases most likely to break assumptions: the first and last site, wrapped and nonwrapped axes, empty neighborhoods, a single agent, zero or maximal allowed counts, and updates that add and remove state in the same timestep. A model that works only away from boundaries has not yet validated its geometry.

14.3 Separate API correctness from scientific validity

Safe-mode checks and passing unit tests show that operations respect tested contracts. They do not establish that the chosen growth law, diffusion coefficient, update schedule, stochastic distribution, or boundary condition is scientifically justified. Validate the numerical method against known solutions where possible; test convergence as grid spacing and timestep change; and compare stochastic results across independent seeds.

Document assumptions alongside the model: whether agents update sequentially or synchronously, whether boundaries are reflecting, fixed, or periodic, whether rates are per unit time or per step, and whether counts or fields represent densities or totals. These assumptions determine what the simulation means.

15. Where to go next

Use the Cheatsheet for short code patterns, the API Guide for behavioral contracts, and the API Reference for current signatures. The `Examples/` directory contains complete models. The Testing guide describes regression coverage and how to run the test suite.